

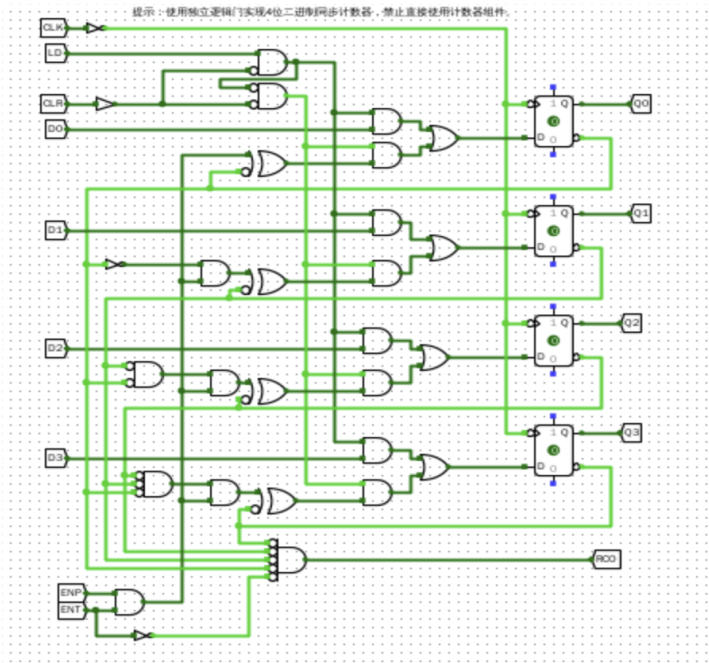
实验三 同步时序部件设计

实验 3.1 计数器实验

1.整体设计思路：

利用 D 触发器和逻辑门电路实现清零，载入，计数的功能。

2.电路图：

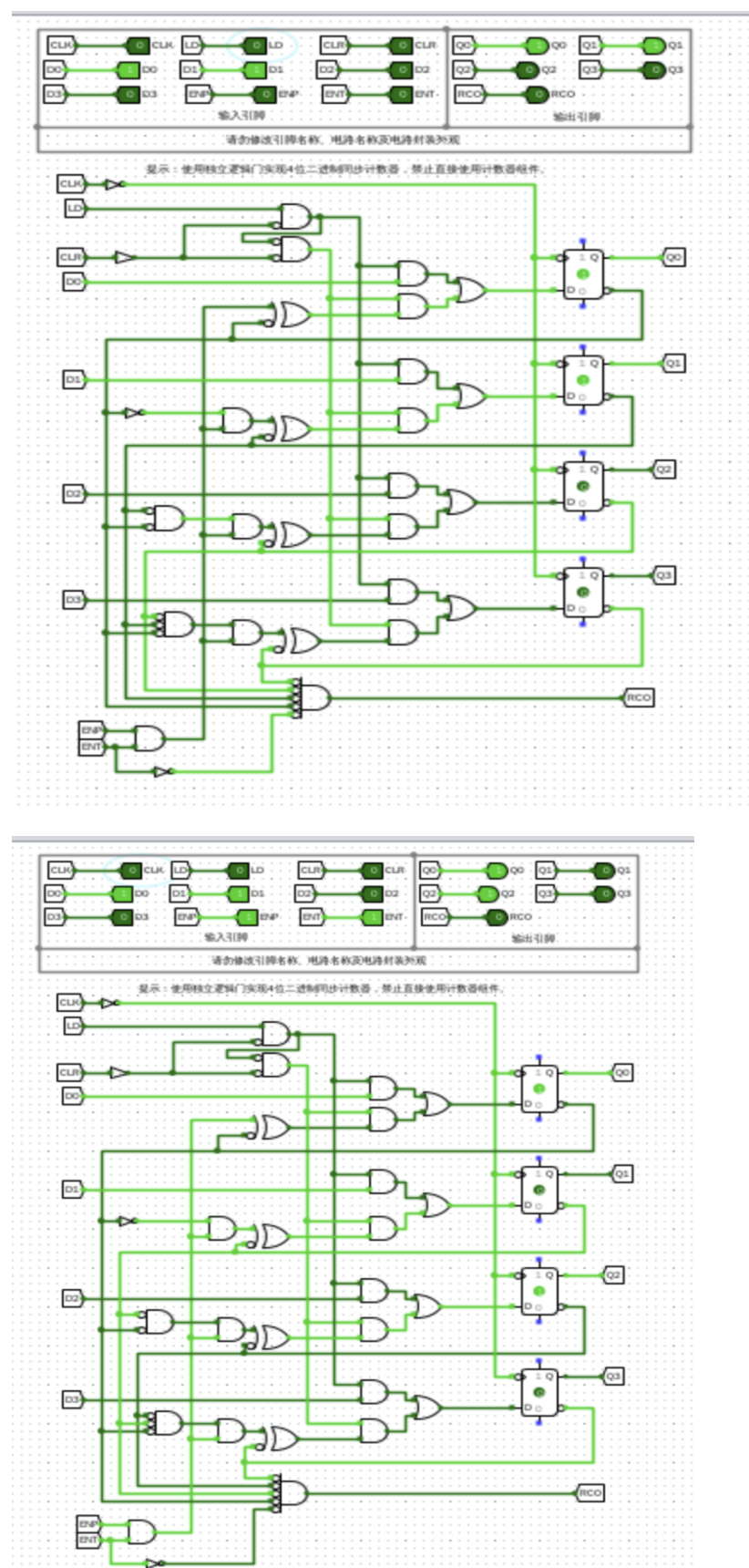


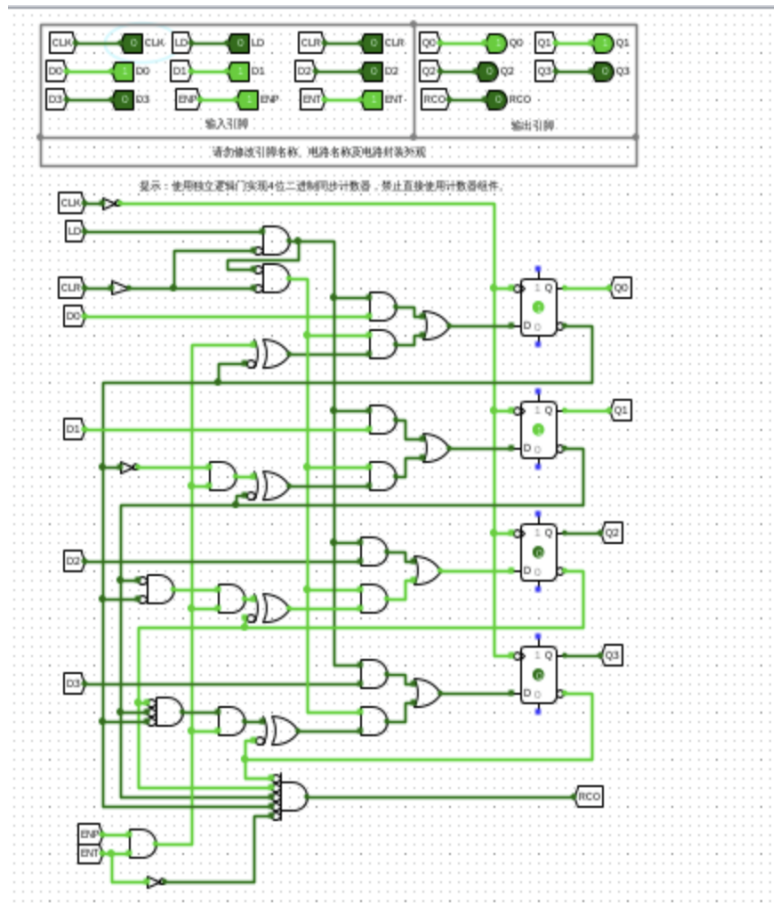
3.功能表：

表 3.1 4 位同步二进制计数器功能表

Inputs				Current State				Next State			
CLR	LD	ENT	ENP	Q3	Q2	Q1	Q0	Q3*	Q2*	Q1*	Q0*
1	x	x	x	x	x	x	x	0	0	0	0
0	1	x	x	x	x	x	x	D3	D2	D1	D0
0	0	0	x	x	x	x	x	Q3	Q2	Q1	Q0
0	0	x	0	x	x	x	x	Q3	Q2	Q1	Q0
0	0	1	1	0	0	0	0	0	0	0	1
0	0	1	1	0	0	0	1	0	0	1	0
...											
0	0	1	1	1	1	0	1	1	1	1	0
0	0	1	1	1	1	1	0	1	1	1	1
0	0	1	1	1	1	1	1	0	0	0	0

4.仿真模拟：





5.错误现象：

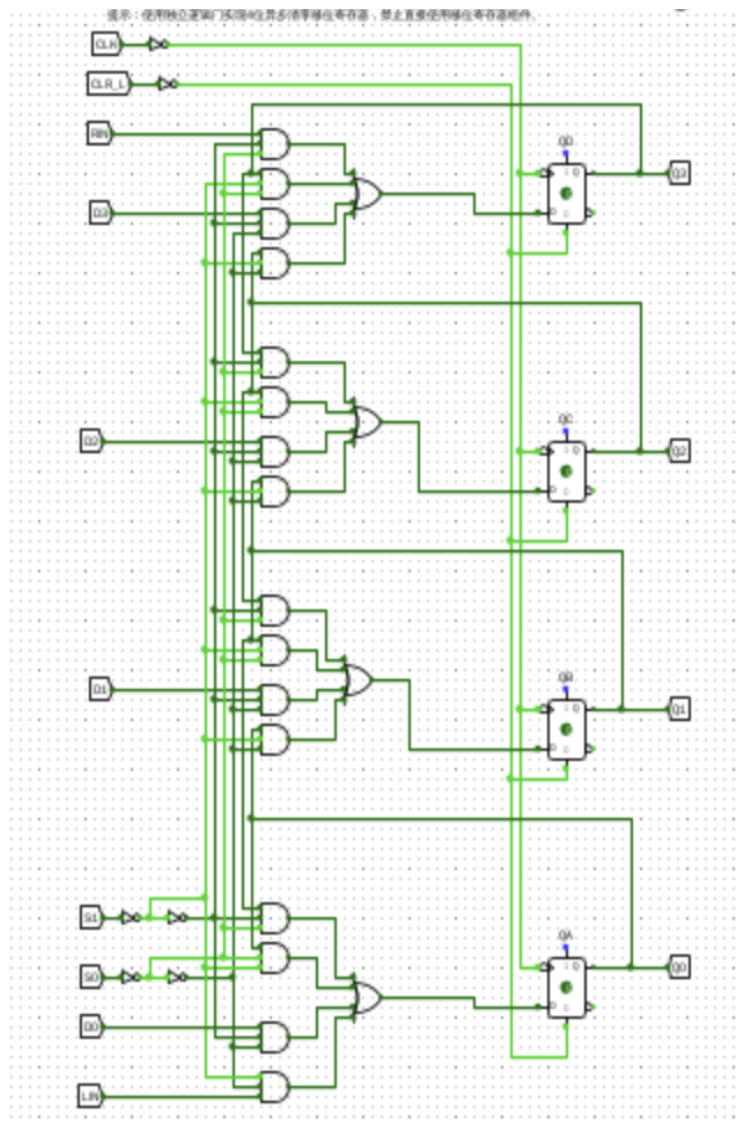
在完成实验的过程中，没有遇到任何错误

实验 3.2 移位寄存器

1.整体设计思路：

通过 D 出发器和逻辑门，利用 S1S0 的值选择并完成载入，左移，右移的功能。

2.电路图：

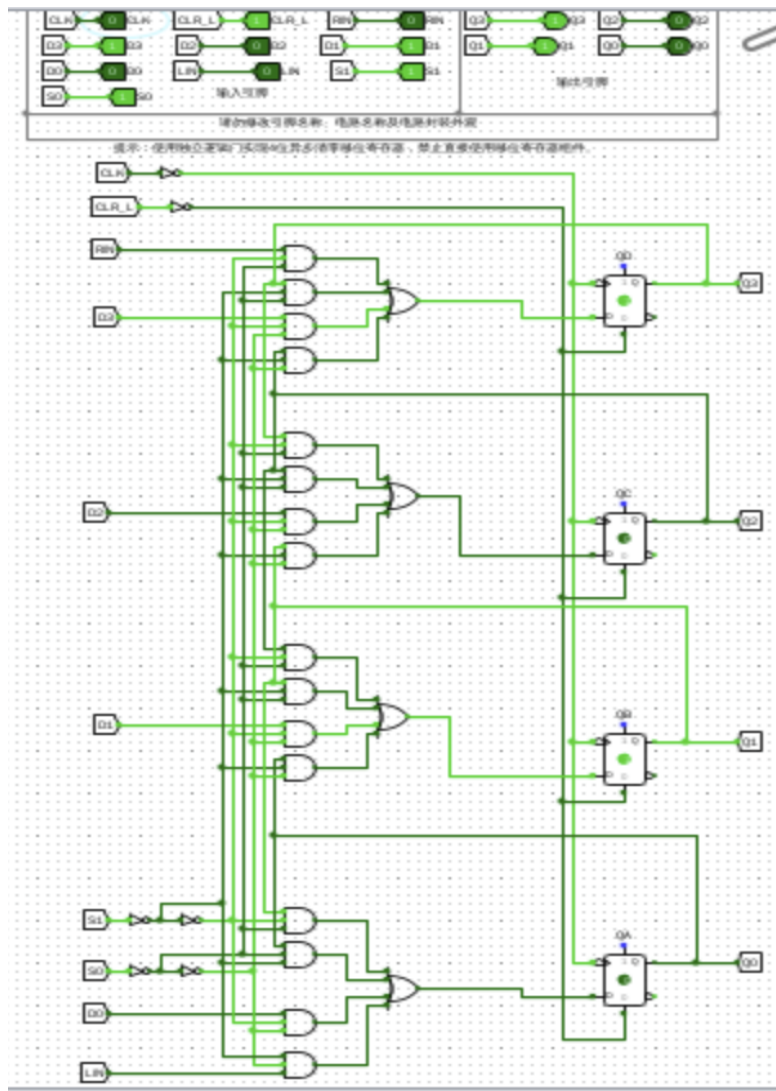


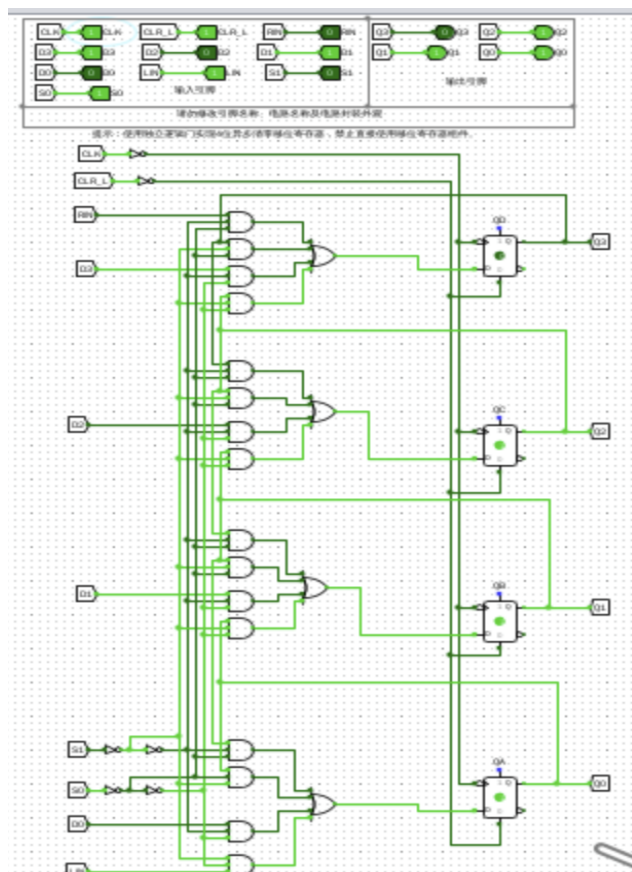
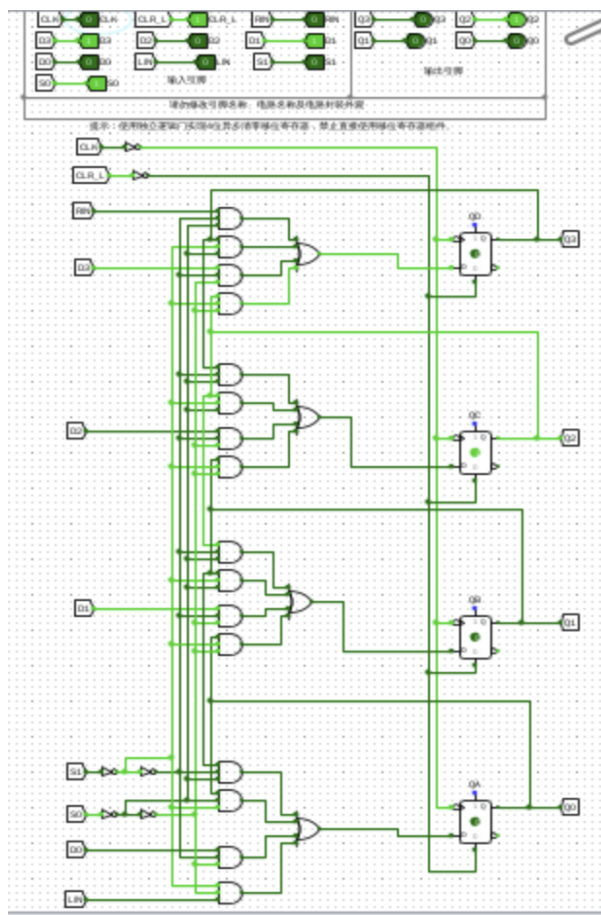
3.功能表：

表 3.2 4 位移位寄存器功能表

功能	输入			下一个状态			
	CLR	S1	S0	Q3*	Q2*	Q1*	Q0*
清零	0	x	x	0	0	0	0
保持	1	0	0	Q3	Q2	Q1	Q0
右移	1	1	0	RIN	Q3	Q2	Q1
左移	1	0	1	Q2	Q1	Q0	LIN
装载	1	1	1	D3	D2	D1	D0

4.仿真模拟：





实验 3.3 4 位无符号数乘法器

1. 整体设计思路：

1) 分别利用两个寄存器储存部分积和乘数，使用 2 选 1 选择器，根据乘数当前位数判断是否要将被乘数送入加法器，随后将加法器和乘数合成 8 位二进制数后送入桶形移位器右移 1 位，将可能存在的进位移入，随后再分别存入两个寄存器。同时利用计数器控制运算次数，在刚开始时利用 load 信号将乘数和 0 载入两个寄存器，在执行 5 次操作（1 次载入，4 次运算）后，停止计算，以精确得出结果。

2) 子模块功能描述：

1. 4 位同步计数器 CNTR4U

功能：控制乘法迭代次数（0~5），为控制逻辑提供计数状态

CLK：时钟输入，驱动计数

CLR：同步清零，复位计数器

LD：同步置数，加载初始值

ENT/ENP：计数使能

D[3:0]：四位输入

Q3~Q0：4 位计数输出

RCO：进位输出，判断计数结束

2. 8 位桶形移位器（修改版）

功能：对部分积、乘数执行右移；新增移入位 IN，替代常量 0

IN：动态移入位

数据输入：8 位待移位数据

Shamt[3:0]：选择右移位数

A/L：控制逻辑/算数移位

L/R：控制左移/右移

数据输出：移位后结果

3.4 位加法器

功能：完成部分积与被乘数的无符号加法

A [3:0]：操作数 A

B [3:0]：操作数 B

Cin：进位输入

Sum [3:0]：和输出

Cout：进位输出

4. 被乘数寄存器（4 位）

功能：锁存数据，保持运算全程数值稳定

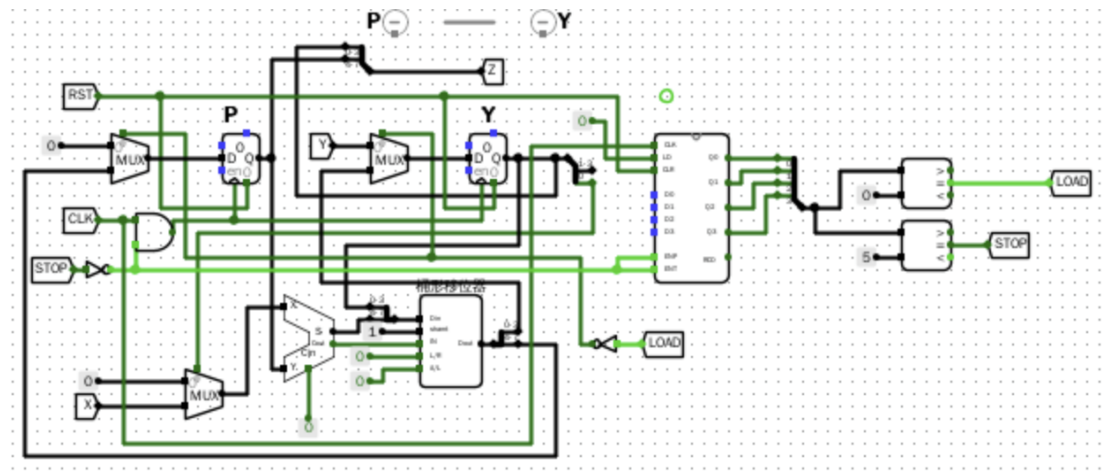
CLK：时钟

D [3:0]：输入

Q [3:0]: 输出

En: 使能端

2.电路图:



3.功能表:

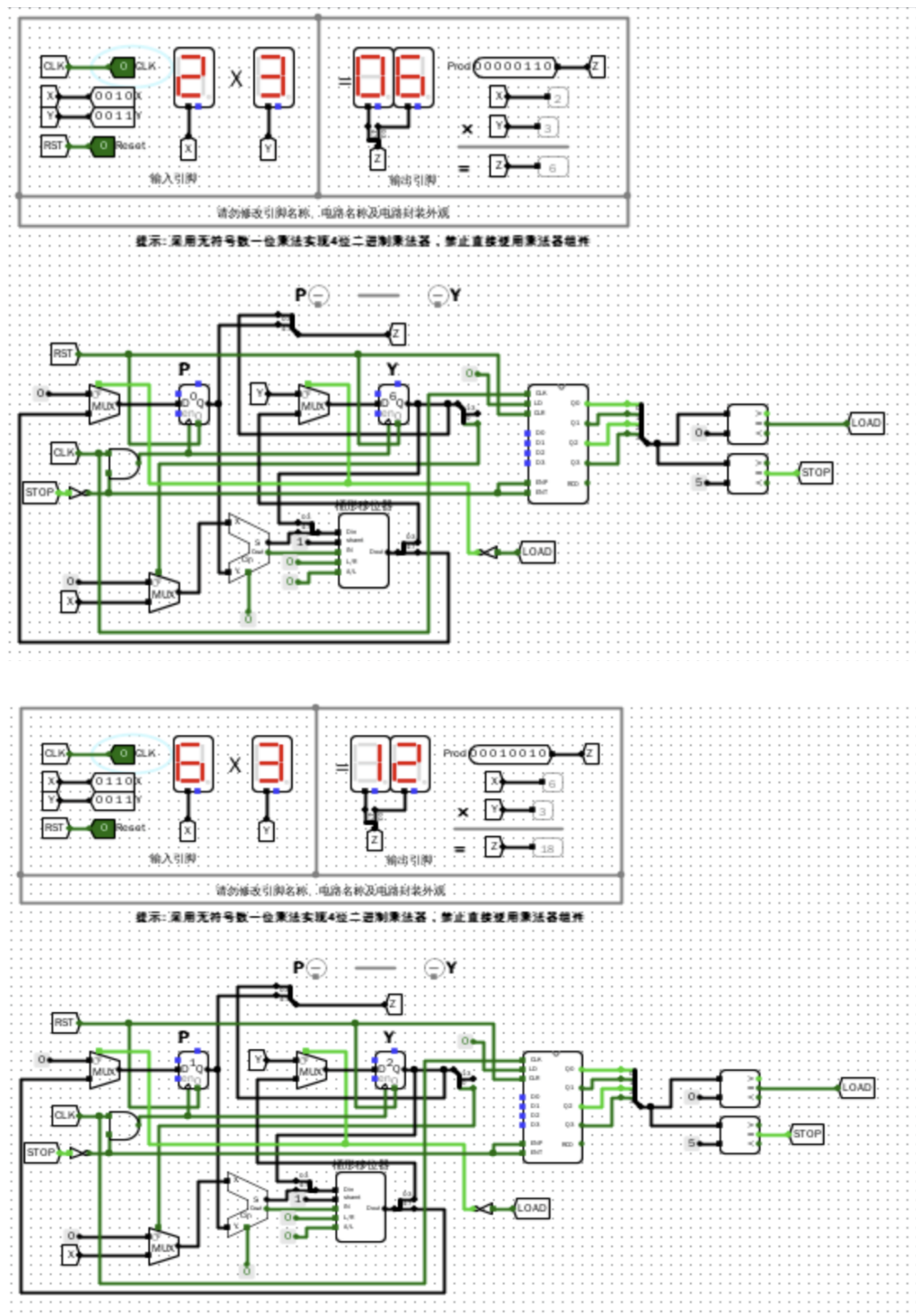
实现乘法功能，输入 X, Y, 和时钟信号 Clk, 复位信号 RST, 得到结果 P, 功能较为简单，且全部数据太过庞大，这里只列举部分。

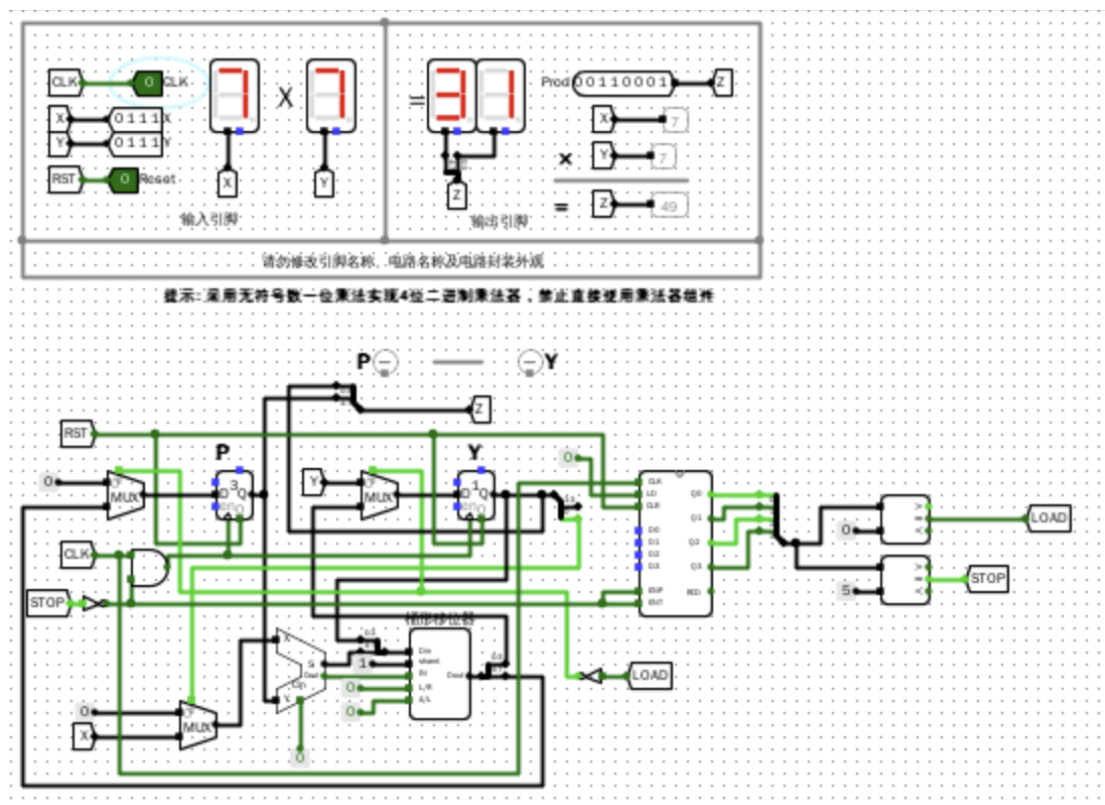
Cnt	Reset	X	Y	Prod
00	1	7	7	00
01	0	7	7	00
02	0	7	7	07
03	0	7	7	3b
04	0	7	7	55
05	0	7	7	62

06 0 7 7 31

07 0 7 7 31

4.仿真模拟:





5. 错误现象及解决方案：

在初次设计的时候，没能正确处理 load 和 stop 信号，导致先是无法正确载入，后来又无法正确执行 4 次计算，在修正了计数器部分的逻辑后，最终成功完成了实验。

实验 3.4 寄存器堆实验

1. 整体设计方案：

通过将 32 个寄存器并联实现寄存器堆，并通过 32to1 译码器控制将数据写入到那个寄存器中，同时用 busW 信号作为写使能，严格控制数据的写入。同时将寄存器的输出分别连入到两个 32 选 1 选择器，用于读取指定位置寄存器中的数据。

2.原理图：

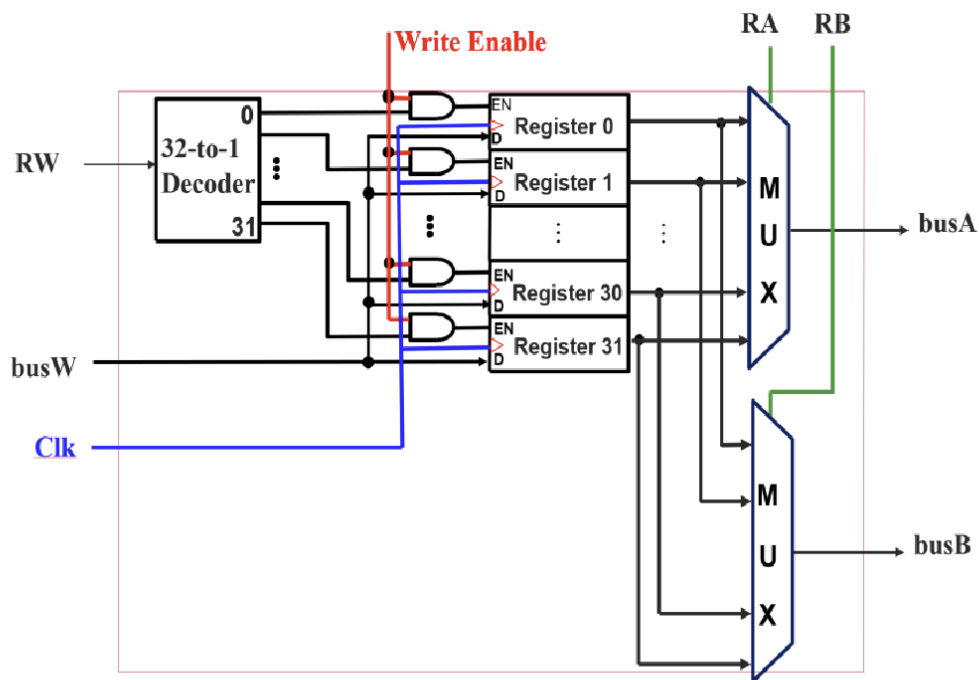


图 3.9 寄存器堆设计原理图

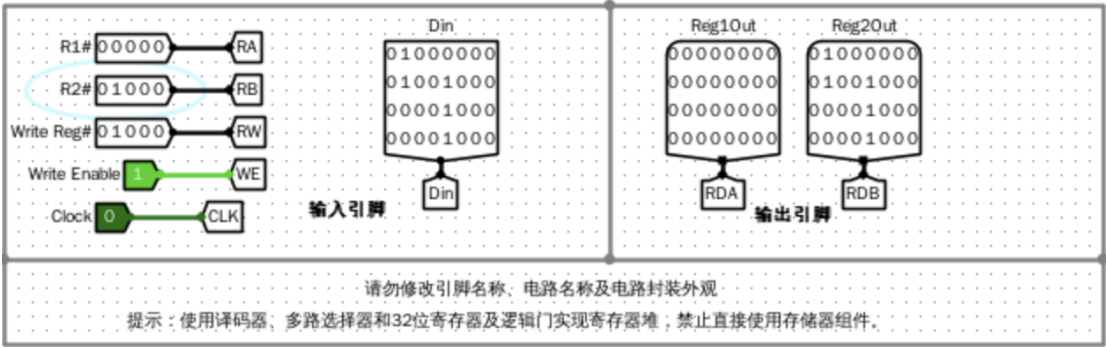
3.功能：

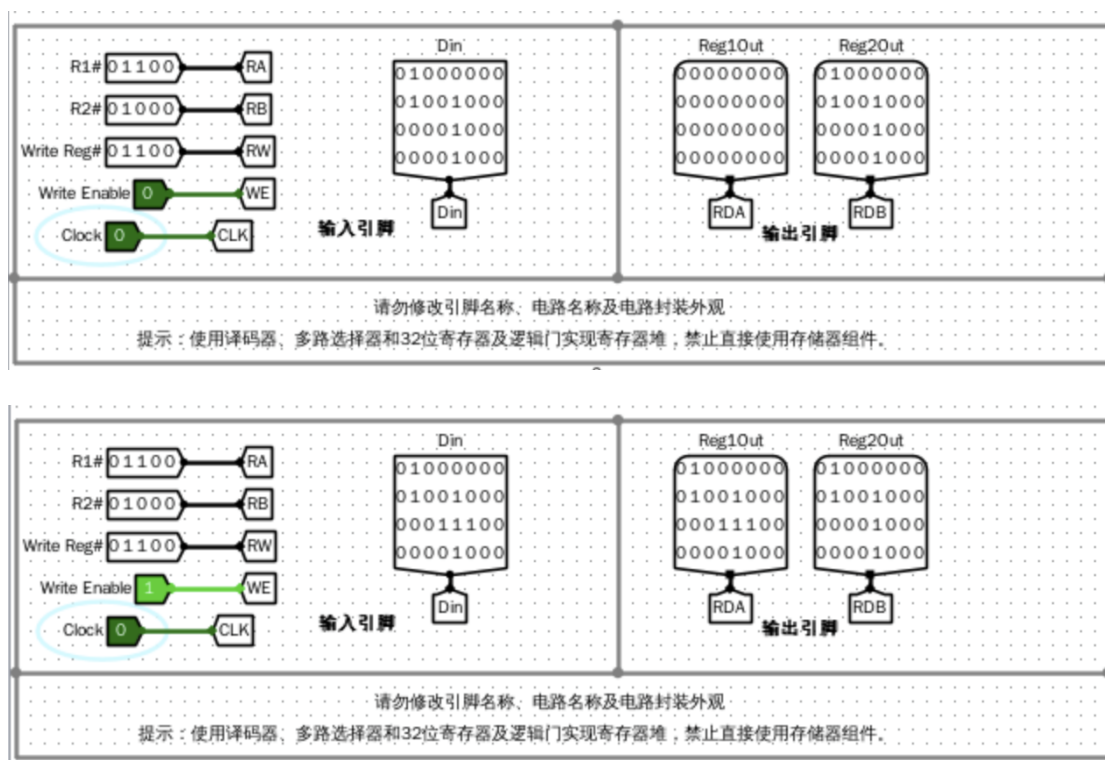
在 busW=1 时，在上升沿将 Din 输入数据写入到由 RW 决定的寄存器

中，同时可以由 RA 和 RB 信号决定，随时将对应寄存器中储存的数据在

RDA 和 RDB 隧道输出。

4.仿真模拟：





5.错误现象：

在完成实验的过程中没有遇到任何错误。

实验 3.5 数字时钟设计

1.整体设计方案：

将整个数字时钟大体分为三部分：

- 第一部分是输入有效判断，当输入不合法并且 LD 载入信号为 1 时，输出 INErr=1
- 第二部分是 LED 信号输出通路，当进入整点后由 LED_EN 脉冲信号“提示”该部分电路工作 10 个时钟周期（主要是依靠 D 触发器将一个短暂的额脉冲信号锁住 10 个时钟周期），并通过逻辑门将计数器的输出转化为格雷码，让 LED 灯颜色按序变化。

C) 最后一部分是时钟计时部分，主题由 6 个 4 为同步计数器组成，当 LD=1 且 INErr=0（试图载入的数据没有错误）时，将时间数据载入到计数器中，同时每个时钟周期视为 1s，在该进位时自动进位/清零。

D) 子模块：

1) 4 位同步计数器：

CLK: 时钟输入（计时脉冲）

CLR: 清零信号

LD: 同步置数（加载初始时间）

ENT、ENP: 计数使能

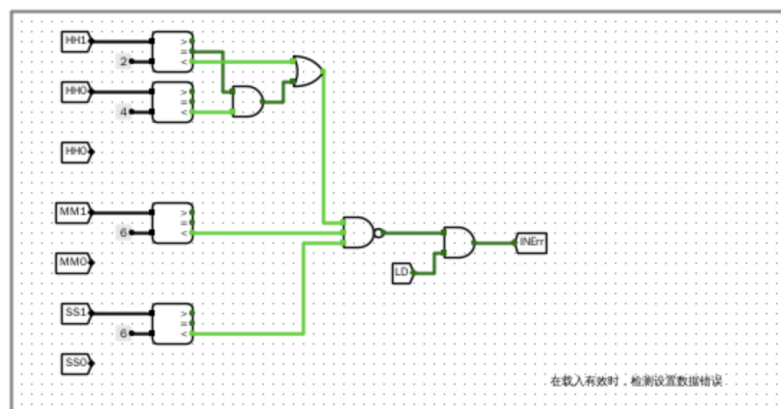
D3~D0: 并行置数输入（BCD 时间初始值）

Q3~Q0: 计数值输出

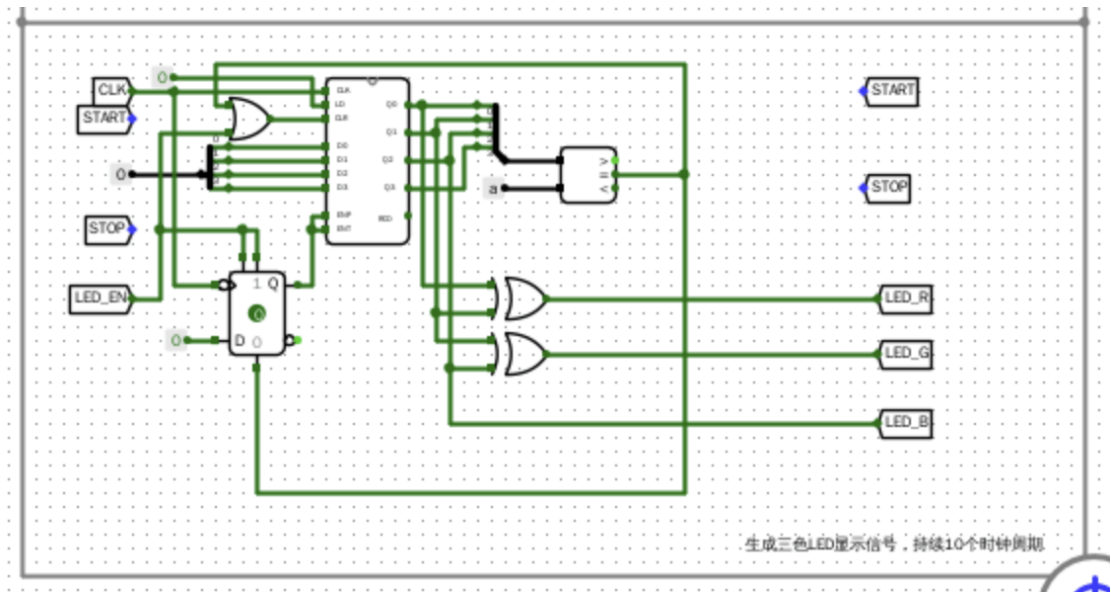
RCO: 进位输出（向高位进位）

2. 电路图：

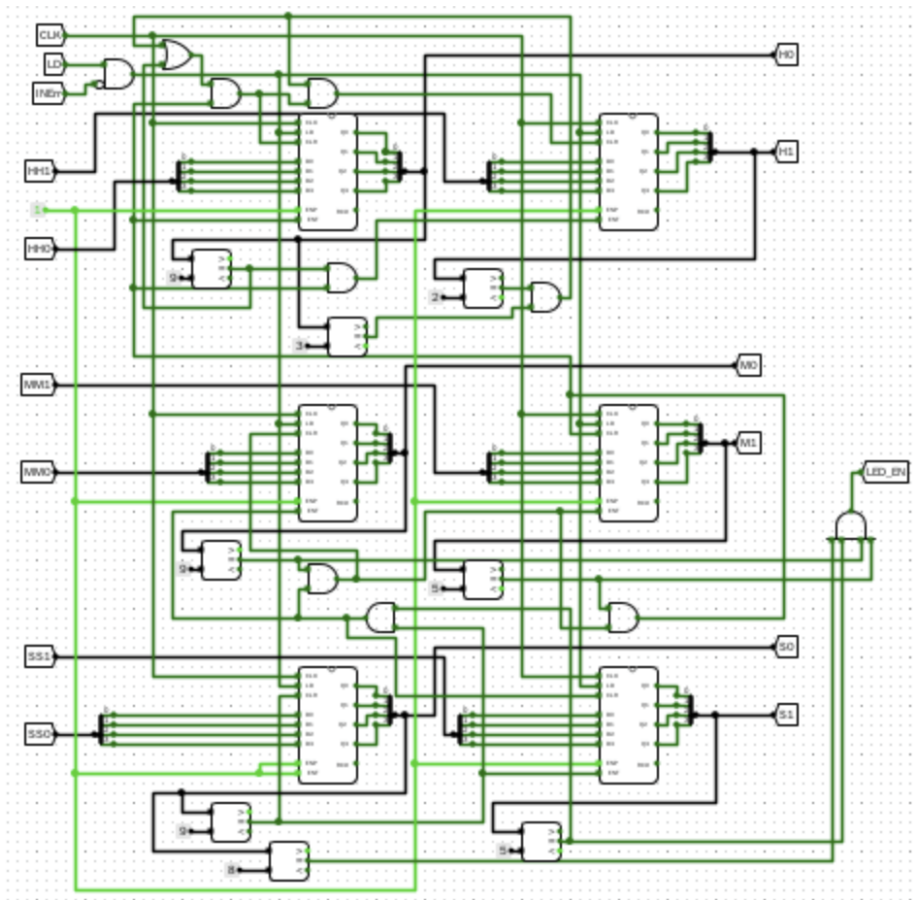
INErr 部分：



LED 信号部分：

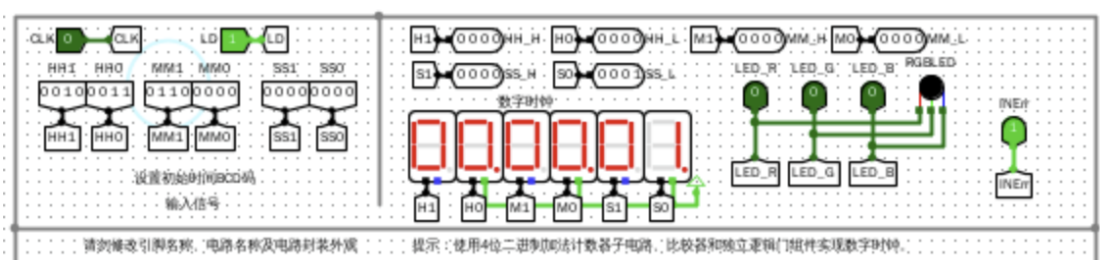


时钟部分：

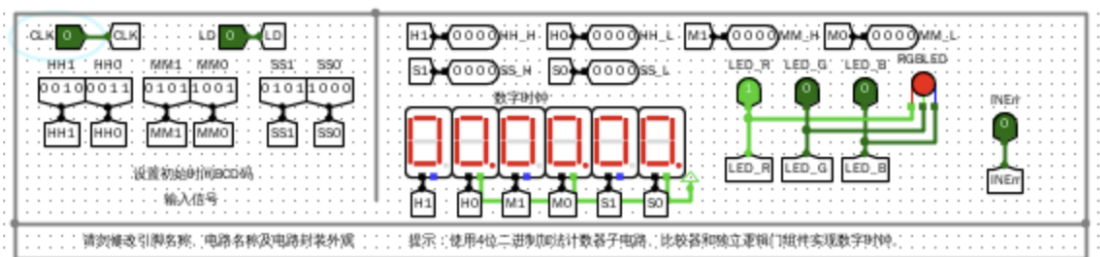
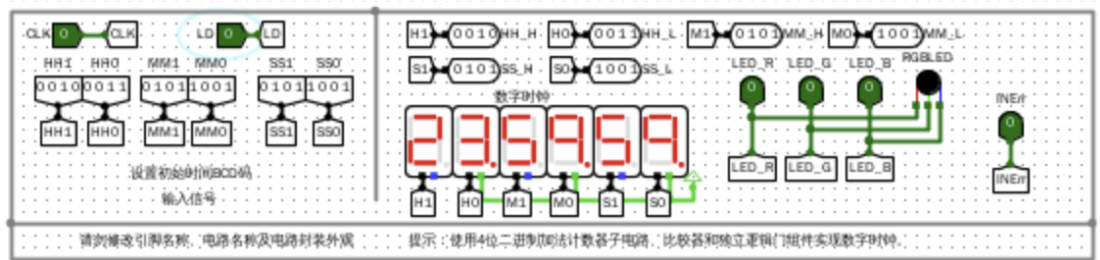


3.功能：

- 1) 每个时钟周期记数 1s，自动完成进位与清零。
 - 2) 在 LD=1 且试图载入的数据无误时，将时间数据载入到寄存器中。
 - 3) 在整点时，让 LED 灯按照红色、黄色、绿色、青色、白色、品红、蓝色、黑色、红色、黄色顺序点亮 10 个时钟周期，随后自动熄灭。
- 4.仿真模拟：



(上图中数据错误，INErr=1 符合预期)



(正常进位并点亮 LED 灯)

5.错误现象与解决方案：

- 1) 一开始没有正确领会题意，导致 LD=0 时 INErr 有时会为 1，通过将原

有 INErr 信号与 LD 与操作后输出即可解决。

2) LED_EN 信号为一个脉冲信号，无法维持 10s，导致 LED 不能被正确点亮。解决方案：通过 D 触发器锁住 LED_EN 信号，将 D 触发器的输出连接到计数器的 ENT 和 ENP 端，同时将 D 触发器的清零端连到计数器输出与 10 比较的“=”输出端以实现 10s 后自动停止记数并清零。

3) 一开始没能争取处理时钟记数以及进位和清零的正确逻辑。

解决方案：秒个位的时钟直接连接 Clk 信号，清零为记数到 9.

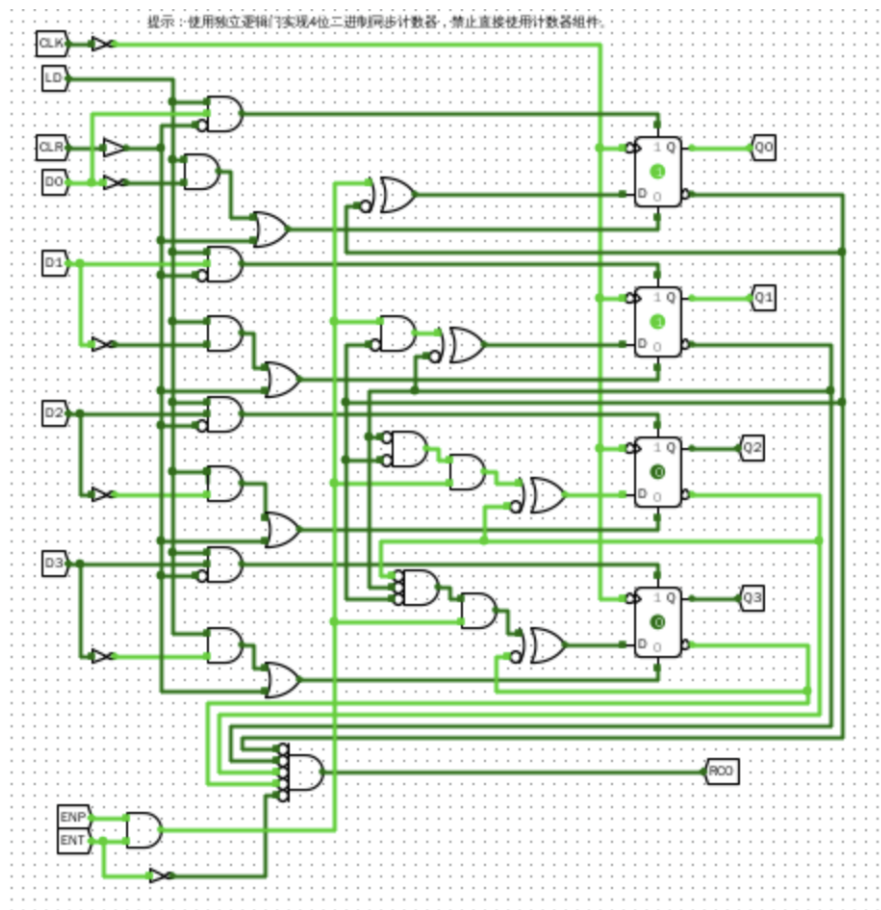
此后每一个计数器的时钟信号是前一个的清零信号，清零信号是自己的时钟信号 AND 前一个的清零信号。

(时十位和时个位的清零较为复杂，需兼具“x9: 59: 59”和“23: 59: 59”的情况)

思考题：

1、修改计数器以实现异步清零和异步置数：

1) 电路图：



2) 设计思路：

由于需要异步清零和异步置数，考虑使用 D 触发器的清零端（下用 R 端代指）和置位端（下用 S 端代指）。

其中置位和清零端的载入逻辑表达式如下：

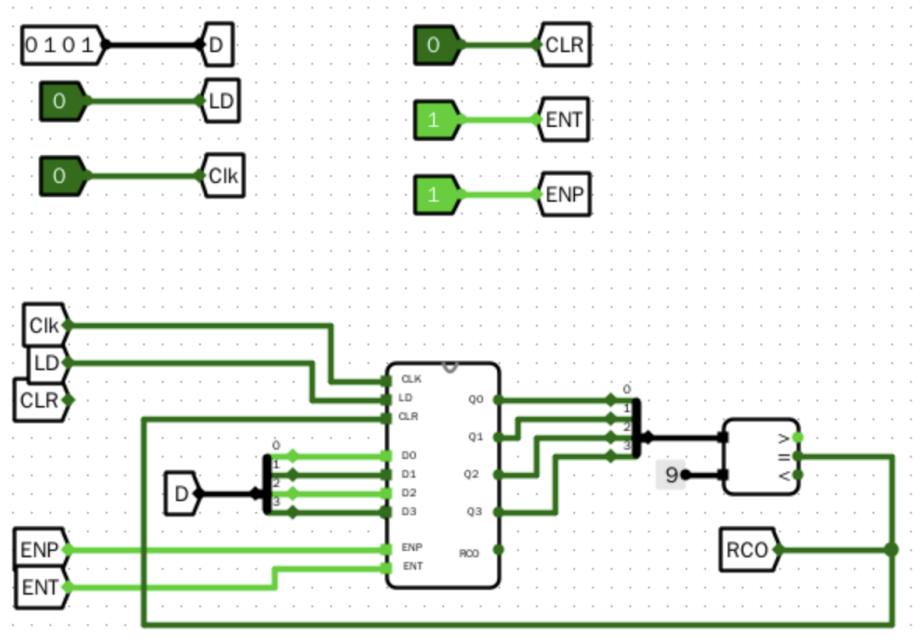
$$R_i = \text{CLR or } (\text{LD and not } D_i)$$

$$S_i = D_i \text{ and LD and not CLR}$$

在正常的记数阶段，采用和正常的同步计数器一样的思路，利用与门和异或门，控制正常加法记数。

2、利用 4 位同步加法计数器实现模 10 计数器：

1) 电路图：

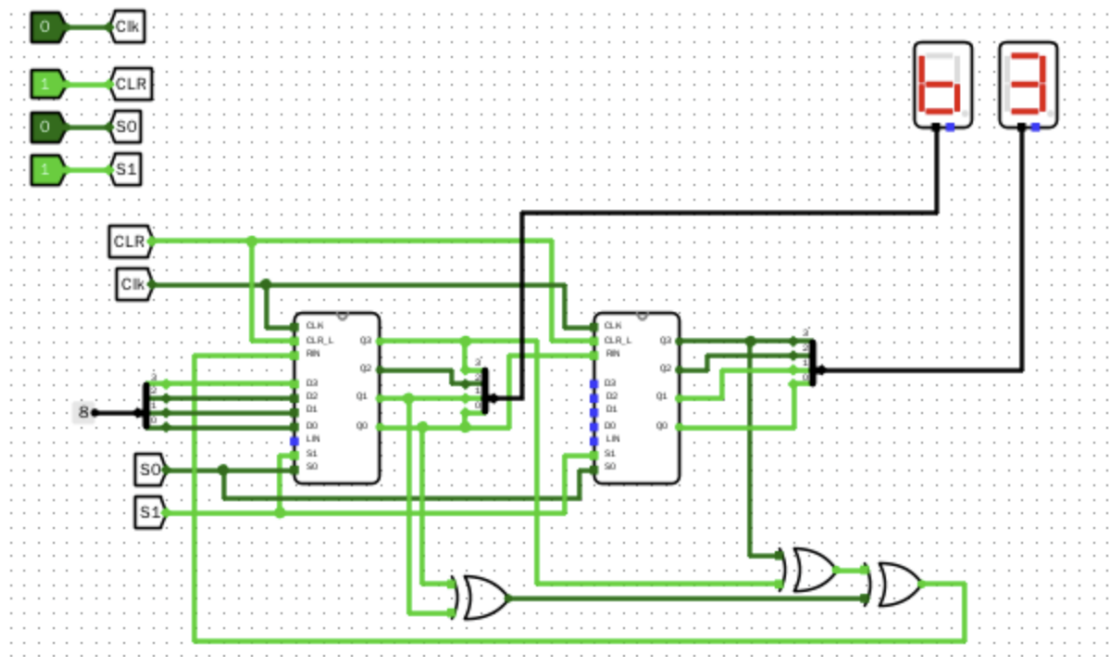


2) 设计思路：

在要求不使用逻辑门，所以考虑使用比较器检测计数器计数情况，当记数到 9 的时候，将 CLR 信号设为 1，这样就能保证记数在 0-9 循环。（由于题目要求不使用逻辑门，暂未想到如何使用 CLR 信号手动清零，只能在计数到 9 时自动清零）

3、利用 4 位移位寄存器实现 8 位二进制伪随机序列电路：

1) 电路图：



2) 设计思路:

利用反馈多项式 $X^8 + X^6 + X^5 + X^4 + 1$ ，将两个移位寄存器串联，不停左移，第一个寄存器的最低位移入第二个寄存器，第二个寄存器的最低位移出，同时将 $Din = Q_8 \oplus Q_6 \oplus Q_5 \oplus Q_4$ 移入第一个寄存器的最高位。为了防止自锁，将初始值设置为 10000000. 这样电路就能从 0x80 开始遍历 0x01 到 0xff 之间的所有数，最后回到 0x80.

起始 10 次数值（全体 255 次的序列太过庞大，不在此全部列出）:

80 -> C0 -> E0 -> 70 -> 38 -> 9C -> CE -> 67 -> B3 -> D9 ->

4、实现寄存器堆的写后读功能:

使用比较器和多路选择器，在写入的位置和试图读取的位置一样时，通过比较器发现异常，然后使用多路选择器直接把要写入的数据同时传输到读取端。